



ВЕЛИКОТЪРНОСВИ УНИВЕРСИТЕТ „СВ. СВ. КИРИЛ И МЕТОДИЙ“
ФАКУЛТЕТ „МАТЕМАТИКА И ИНФОРМАТИКА“



КУРСОВА РАБОТА

ПО
учебната дисциплина
СИСТЕМНО ПРОГРАМИРАНЕ

Тема: „Система за пешеходен светофар с бутон, таймер,
звуков сигнал и OLED дисплей“

ИЗГОТВИЛ:

Мирослава Красиминова Пенева –
фак. № 2209011272
Спец. „Компютърни науки”, III курс

ПРОВЕРИЛ:

проф. д-р Мирослав Николов
Гълъбов
Мирослав Бончев Бончев

*Велико Търново
2024 - 2025 г.*

СЪДЪРЖАНИЕ

1	Същност	3
2	Алгоритъм.....	4
3	Реализация.....	6
4	Заключение и бъдеща работа	16

1 Същност

Проектът представлява интелигентна система за управление на пешеходна и автомобилна светофарна уредба, реализирана върху Raspberry Pi с помощта на GPIO пинове и OLED дисплей с I2C интерфейс (SH1106). Основната цел е да се осигури безопасно преминаване на пешеходци, като системата реагира на натискане на бутон за пешеходна заявка, управлява LED светлини (червено, жълто, зелено за автомобили и пешеходци), активира звуков сигнал и визуализира обратното броене на OLED дисплея.

Особено внимание е отделено на зумера, който служи като аудио сигнал за незрящи пешеходци, осигурявайки им допълнителна информация кога е безопасно да пресекат, чрез звукови индикации по време на обратното броене.

Кодът е организиран с многониткова архитектура и съдържа отделни нишки за:

- Контрол на светофарната последователност и активиране на пешеходна последователност с аудио и визуална индикация.
- Наблюдение на състоянието на Ethernet връзката, като при прекъсване системата преминава в безопасен режим и спира работа.

Обработката на натискането на бутона се извършва чрез хардуерна обработка на прекъсване (ISR), като е имплементирана анти-дребезгова логика и дебаунс контрол.

OLED дисплеят се инициализира и използва за изобразяване на големи цифрови символи, които отброяват оставащото време за преминаване на пешеходеца. Звуковият сигнал на зумера се активира по време на обратното броене, което допринася за по-голяма достъпност на системата за хора с увредено зрение.

Сигналите за прекъсване (например Ctrl+C) се обработват за коректно изключване — гасене на всички светлини, спиране на звуковия сигнал и изчистване на дисплея, осигурявайки безопасно и чисто приключване на програмата.

Тази реализация демонстрира ефективна синхронизация между хардуерните компоненти и многонитковата логика, вградена в Linux среда, като позволява плавно и надеждно управление на пешеходната светофарна система с обратна връзка за състоянието на мрежата и подобрена достъпност за незрящи.

2 Алгоритъм

1. Инициализация:

- Настройване на GPIO пиновете (за светлини, бутон, зумер).
- Инициализиране на I2C връзката с OLED дисплея.
- Включване на начално състояние на светлините:
 - Зелено за автомобили.
 - Червено за пешеходци.
- Изчистване и включване на дисплея.
- Стартиране на две нишки:
 - Нишка за Ethernet наблюдение.
 - Нишка за контрол на светофара.

2. Обработка на бутона:

- Обработка се чрез хардуерна функция за прекъсване (ISR) с анти-дребезгова логика.
- При натискане на бутона:
 - Ако няма активна пешеходна заявка, таймерът не работи и Ethernet връзката е активна:
 - Задава `pedestrian_request = true` и `timer_running = true`.
 - Сигнализира главната нишка за новата заявка.

3. Основен цикъл (нишка `trafficLightController`):

- Изчаква сигнал за пешеходна заявка.
- При получаване на заявка изпълнява пешеходна последователност.

4. Пешеходна последователност (`pedestrianSequence`):

- Изчакване 5 секунди (буфер преди спиране на автомобилния трафик).
- Превключване на светофара:
 - Изгасване на зелено за автомобили.
 - Включване на жълто за 2 секунди.
 - Включване на червено за автомобили и изчакване 2 секунди.
- Пешеходците преминават:
 - Изгасване на червено за пешеходци.
 - Включване на зелено за пешеходци.
 - Активиране на зумера (за незрящи пешеходци).
 - Стартиране на 20-секундно обратно броене на OLED дисплея.
- Край на преминаването:
 - Изгасване на зелено за пешеходци.
 - Включване на червено за пешеходци.
 - Изчакване 5 секунди.
- Възстановяване на автомобилния поток:
 - Включване на жълто за 2 секунди.
 - Включване на зелено за автомобили.
 - Изгасване на жълто и червено.
- Изключване на `timer_running`.

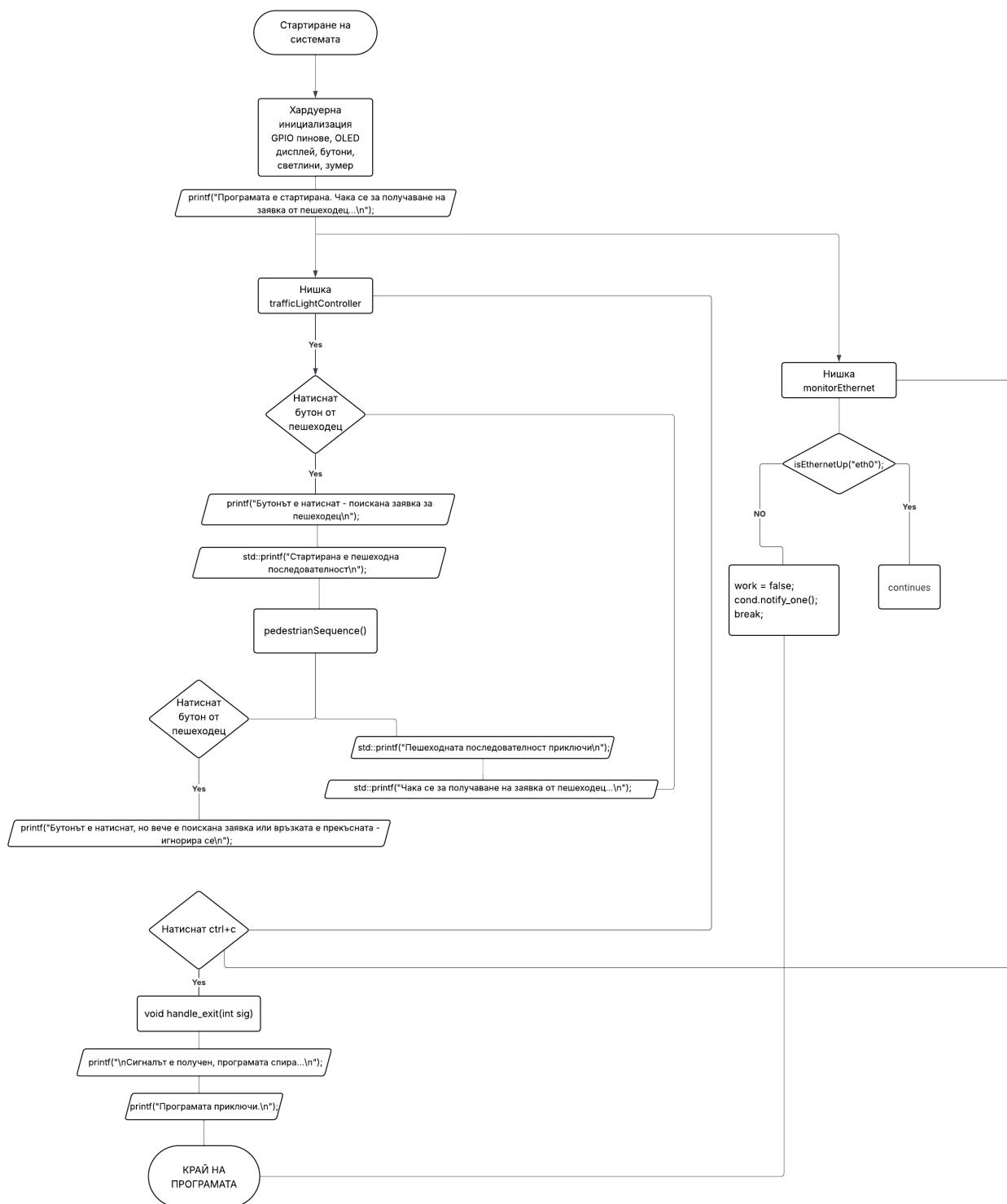
5. Нишка за Ethernet наблюдение (`monitorEthernet`):

- Проверява състоянието на мрежовия интерфейс `eth0` на всяка секунда.
- При промяна на състоянието отпечатва съответно съобщение за свързаност или прекъсване.
- Ако Ethernet е прекъснат и няма активен таймер:
 - Системата се изключва незабавно.
- Ако има активен таймер (пешеходна последователност):
 - Изчаква да завърши последователността и после спира системата.

6. Изход и почистване (`handle_exit`):

- При сигнал за прекъсване (например Ctrl+C):

- Изключва всички светлини и зумера.
- Изчиства и изключва дисплея.
- Прекратява програмата безопасно.



3 Реализация

```
#include <csignal>
```

```
#include <unistd.h>
```

```
#include <atomic>
```

```
#include <thread>
```

```
#include <mutex>
```

```
#include <condition_variable>
```

```
#include <wiringPi.h>
```

```
#include <wiringPiI2C.h>
```

```
#include <cstdio>
```

```
#include <chrono>
```

```
#include <fstream>
```

```
#include <cstdlib>
```

```
constexpr int CAR_GREEN    = 17;
```

```
constexpr int CAR_YELLOW   = 27;
```

```
constexpr int CAR_RED      = 22;
```

```
constexpr int PED_RED      = 25;
```

```
constexpr int PED_GREEN    = 16;
```

```
constexpr int BUTTON_PIN   = 26;
```

```
constexpr int BUZZER_PIN   = 21;
```

```
constexpr int SH1106_I2C_ADDR = 0x3C;
```

```
int fd = -1;
```

```
bool work = true;
```

```
bool pedestrian_request = false;
```

```
bool timer_running = false;
```

```
bool ethernet_connected = true;
```

```
std::mutex mutex;
```

```
std::condition_variable cond;
```

Системно програмиране, Система за пешеходен светофар с бутон, таймер, звуков сигнал и OLED дисплей

```
void sendCommand(uint8_t cmd) {
```

```
    wiringPiI2CWriteReg8(fd, 0x00, cmd);
```

```
}
```

```
void sendData(uint8_t data) {
```

```
    wiringPiI2CWriteReg8(fd, 0x40, data);
```

```
}
```

```
void initDisplay() {
```

```
    sendCommand(0xAE);
```

```
    sendCommand(0xD5); sendCommand(0x80);
```

```
    sendCommand(0xA8); sendCommand(0x3F);
```

```
    sendCommand(0xD3); sendCommand(0x00);
```

```
    sendCommand(0x40);
```

```
    sendCommand(0xAD); sendCommand(0x8B);
```

```
    sendCommand(0xA1);
```

```
    sendCommand(0xC8);
```

```
    sendCommand(0xDA); sendCommand(0x12);
```

```
    sendCommand(0x81); sendCommand(0xCF);
```

```
    sendCommand(0xD9); sendCommand(0xF1);
```

```
    sendCommand(0xDB); sendCommand(0x40);
```

```
    sendCommand(0xA4);
```

```
    sendCommand(0xA6);
```

```
    sendCommand(0xAF);
```

```
}
```

```
void clearDisplay() {
```

```
    for (int page = 0; page < 8; ++page) {
```

```
        sendCommand(0xB0 + page);
```

```
        sendCommand(0x00);
```

```
        sendCommand(0x10);
```

```
        for (int col = 0; col < 132; ++col) {
```

```
        sendData(0x00);

    }

}

void turnOffDisplay() {
    sendCommand(0xAE);
}

void setCursor(int page, int col) {
    sendCommand(0xB0 + page);
    sendCommand(0x00 + (col & 0x0F));
    sendCommand(0x10 + ((col >> 4) & 0x0F));
}

const uint8_t bigDigits16x8[10][16] = {
    {0x3C,0x00,0x42,0x00,0x81,0x00,0x81,0x00,0x81,0x00,0x81,0x00,0x42,0x00,0x3C,0x00},
    {0x00,0x00,0x84,0x00,0xFE,0x00,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
    {0xC2,0x00,0xA1,0x00,0x91,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x86,0x00},
    {0x42,0x00,0x81,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x76,0x00},
    {0x30,0x00,0x28,0x00,0x24,0x00,0x22,0x00,0xFE,0x00,0x20,0x00,0x20,0x00,0x00,0x00},
    {0x4F,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x71,0x00},
    {0x7E,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x72,0x00},
    {0x01,0x00,0xE1,0x00,0x11,0x00,0x09,0x00,0x05,0x00,0x03,0x00,0x01,0x00,0x00,0x00},
    {0x76,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x76,0x00},
    {0x06,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x89,0x00,0x49,0x00,0x3E,0x00,0x00}
};

void drawBigDigit16x8(int page, int col, int digit) {
    if (digit < 0 || digit > 9) return;
    setCursor(page, col);
    for (int i = 0; i < 8; ++i) sendData(bigDigits16x8[digit][i * 2]);
}
```


Системно програмиране, Система за пешеходен светофар с бутон, таймер, звуков сигнал и OLED дисплей

```
setCursor(page + 1, col);

for (int i = 0; i < 8; ++i) sendData(bigDigits16x8[digit][i * 2 + 1]);
}

void countdownTimer(int seconds) {
    digitalWrite(BUZZER_PIN, HIGH);

    for (int i = seconds; i >= 0 && work && timer_running; --i) {
        printf("Оставащи секунди: %d\n", i);
        for (int page = 3; page <= 4; ++page) {
            setCursor(page, 44);
            for (int col = 44; col < 60; ++col) sendData(0x00);
        }

        int tens = i / 10;
        int ones = i % 10;
        drawBigDigit16x8(3, 44, tens);
        drawBigDigit16x8(3, 52, ones);

        for (int ms = 0; ms < 1000 && work && timer_running; ms += 100) {
            usleep(100 * 1000);
        }
    }

    digitalWrite(BUZZER_PIN, LOW);
    clearDisplay();
}

void pedestrianSequence() {
    std::printf("Стартирана е пешеходна последователност\n");

    std::this_thread::sleep_for(std::chrono::seconds(5));
```

```
if (!work) return;
```

```
digitalWrite(CAR_GREEN, LOW);
```

```
digitalWrite(CAR_YELLOW, HIGH);
```

```
std::this_thread::sleep_for(std::chrono::seconds(2));
```

```
if (!work) return;
```

```
digitalWrite(CAR_YELLOW, LOW);
```

```
digitalWrite(CAR_RED, HIGH);
```

```
std::this_thread::sleep_for(std::chrono::seconds(2));
```

```
if (!work) return;
```

```
digitalWrite(PED_RED, LOW);
```

```
if (!work) return;
```

```
digitalWrite(PED_GREEN, HIGH);
```

```
countdownTimer(20);
```

```
if (!work) return;
```

```
digitalWrite(PED_GREEN, LOW);
```

```
digitalWrite(PED_RED, HIGH);
```

```
std::this_thread::sleep_for(std::chrono::seconds(5));
```

```
if (!work) return;
```

```
digitalWrite(CAR_RED, LOW);
```

```
digitalWrite(CAR_YELLOW, HIGH);
```

```
std::this_thread::sleep_for(std::chrono::seconds(2));
```

```
if (!work) return;
```

```
digitalWrite(CAR_YELLOW, LOW);
```

```
digitalWrite(CAR_GREEN, HIGH);
```

```
{  
    std::lock_guard<std::mutex> lock(mutex);  
    pedestrian_request = false;  
    timer_running = false;  
}  
  
std::printf("Пешеходната последователност приключи\n");  
}  
  
void trafficLightController() {  
    while (work) {  
        std::unique_lock<std::mutex> lock(mutex);  
        cond.wait(lock, [] { return pedestrian_request || !work; });  
        if (!work) break;  
        pedestrian_request = false;  
        lock.unlock();  
  
        pedestrianSequence();  
  
        std::lock_guard<std::mutex> lock2(mutex);  
        if (!ethernet_connected) {  
            work = false;  
            cond.notify_one();  
            break;  
        }  
    }  
}  
  
bool isEthernetUp(const std::string& iface) {  
    std::ifstream f("/sys/class/net/" + iface + "/operstate");  
    if (!f.is_open()) {
```

```
    printf("Cannot open operstate for %s\n", iface.c_str());  
  
    return false;  
  
}  
  
std::string state;  
std::getline(f, state);  
f.close();  
return (state == "up");  
}
```

```
void monitorEthernet() {  
    while (work) {  
        bool connected = isEthernetUp("eth0");  
  
        {  
            std::lock_guard<std::mutex> lock(mutex);  
            if (connected != ethernet_connected) {  
                ethernet_connected = connected;  
                if (!ethernet_connected) {  
                    printf("Ethernet прекъснат!\n");  
                } else {  
                    printf("Ethernet свързан!\n");  
                }  
            }  
        }  
  
        if (!ethernet_connected && !timer_running) {  
            work = false;  
            cond.notify_one();  
            break;  
        }  
    }  
  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
}
```

```
}  
}  
  
void handle_exit(int sig) {  
    (void)sig;  
    {  
        std::lock_guard<std::mutex> lock(mutex);  
        work = false;  
    }  
    cond.notify_one();  
  
    digitalWrite(CAR_GREEN, LOW);  
    digitalWrite(CAR_YELLOW, LOW);  
    digitalWrite(CAR_RED, LOW);  
    digitalWrite(PED_RED, LOW);  
    digitalWrite(PED_GREEN, LOW);  
    digitalWrite(BUZZER_PIN, LOW);  
  
    clearDisplay();  
    turnOffDisplay();  
  
    printf("\nСигналят е получен, програмата спира...\n");  
}  
  
void buttonISR() {  
    static std::chrono::steady_clock::time_point last_press_time = std::chrono::steady_clock::now() -  
    std::chrono::milliseconds(1000);  
    auto current_time = std::chrono::steady_clock::now();  
  
    if (std::chrono::duration_cast<std::chrono::milliseconds>(current_time - last_press_time).count() < 300) {  
        return;  
    }  
}
```

```
last_press_time = current_time;

std::lock_guard<std::mutex> lock(mutex);

if (pedestrian_request || timer_running || !ethernet_connected) {
    printf("Бутон е вече натиснат или няма мрежа, игнориране.\n");
    return;
}

pedestrian_request = true;
timer_running = true;
cond.notify_one();

printf("Бутонът е натиснат, започва пешеходна последователност.\n");
}

int main() {
    signal(SIGINT, handle_exit);

    if (wiringPiSetupGpio() == -1) {
        printf("Грешка при инициализация на WiringPi\n");
        return 1;
    }

    printf("Програмата е стартирана. Чака се за получаване на заявка от пешеходец...\n");

    pinMode(CAR_GREEN, OUTPUT);
    pinMode(CAR_YELLOW, OUTPUT);
    pinMode(CAR_RED, OUTPUT);
    pinMode(PED_RED, OUTPUT);
    pinMode(PED_GREEN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT);
    pullUpDnControl(BUTTON_PIN,PUD_UP);
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(CAR_GREEN, HIGH);
```

```
digitalWrite(CAR_YELLOW, LOW);
```

```
digitalWrite(CAR_RED, LOW);
```

```
digitalWrite(PED_RED, HIGH);
```

```
digitalWrite(PED_GREEN, LOW);
```

```
digitalWrite(BUZZER_PIN, LOW);
```

```
fd = wiringPiI2CSetup(SH1106_I2C_ADDR);
```

```
if (fd == -1) {
```

```
    printf("Грешка при инициализация на дисплея\n");
```

```
    return 1;
```

```
}
```

```
initDisplay();
```

```
clearDisplay();
```

```
if (wiringPiISR(BUTTON_PIN, INT_EDGE_FALLING, &buttonISR) < 0) {
```

```
    printf("Грешка при настройка на ISR\n");
```

```
    return 1;
```

```
}
```

```
std::thread trafficThread(trafficLightController);
```

```
std::thread ethernetThread(monitorEthernet);
```

```
trafficThread.join();
```

```
ethernetThread.join();
```

```
digitalWrite(CAR_GREEN, LOW);
```

```
digitalWrite(CAR_YELLOW, LOW);
```

```
digitalWrite(CAR_RED, LOW);
```

```
digitalWrite(PED_RED, LOW);  
  
digitalWrite(PED_GREEN, LOW);  
  
digitalWrite(BUZZER_PIN, LOW);  
  
  
clearDisplay();  
  
turnOffDisplay();  
  
  
printf("Програмата приключи успешно.\n");  
  
  
return 0;  
}
```

4 Заключение и бъдеща работа

Настоящата система успешно реализира основна логика за управление на светофар с възможност за пешеходна заявка и наблюдение на мрежовата свързаност. В бъдеще функционалността може да бъде значително разширена. Някои възможни посоки за развитие включват:

- **Визуализация на допълнителна информация на дисплея** – като време, състояние на системата или мрежова диагностика.
- **Интеграция с интернет платформи (IoT)** – за дистанционно наблюдение и управление чрез мобилно приложение или уеб интерфейс.
- **Събиране и логване на данни** – за анализ на трафик и подобряване на ефективността.
- **Адаптивно управление на времето на светофарите** – въз основа на реално време или сензори за трафик.
- **Захранване с резервна батерия** и автоматично рестартиране при прекъсване на захранването.

С тези подобрения системата може да се превърне в интелигентна и устойчива част от градската инфраструктура, способна да отговаря на изискванията на съвременните „умни“ градове.

<https://github.com/Miroslava2003/Raspberry-Pi-3-mobel-b.git>